

# The University of Azad Jammu & Kashmir, Muzaffarabad

## Department of Software Engineering

# Software Construction & Development (Lab)

## Lab 04

| | |
|---|---|
| **Task Title:** | **Class Modeling, Object-Oriented Inheritance & System Design** |
| **Submitted to:** | **Engr. Dr. Asma Javed** |
| **Student Name:** | **Feeza Kulsoom** |
| **Roll No:** | **2024-SE-03** |
| **Reg Number:** | **2024-UMDB-004738** |

# Task 1: Online Doctor Appointment Management System

**Problem Statement:** Design an Online Doctor Appointment Management System. Patients/Users create accounts to apply for appointments with health specialists. Doctors maintain accounts to review incoming appointment requests, accepting requests if their hospital/clinic schedule is open or rejecting requests if fully booked. Upon acceptance, an automated confirmation is dispatched to the patient. Identify the core classes, detail their attributes and operations, and construct the complete UML Class Diagram.

## 1.1 Domain Classes & Architectural Responsibilities

- **User (Abstract Superclass):** Serves as the generalized base class encapsulating common identity and authentication credentials (userId, fullName, email, password, phoneNumber, address) along with generalized behavioral methods such as register(), login(), logout(), and updateProfile().
- **Patient (Subclass of User):** Specializes the User entity to represent healthcare seekers. Contains medical demographic attributes (medicalHistory, bloodGroup, emergencyContact) and behavioral operations (searchDoctor(), requestAppointment(), cancelAppointment(), viewHistory()).
- **Doctor (Subclass of User):** Specializes the User entity to represent medical practitioners. Contains professional credentials (doctorId, specialization, qualification, clinicAddress, consultationFee) and operational methods (manageSchedule(), acceptAppointment(), rejectAppointment(), viewPatientHistory()).
- **Appointment (Association Entity):** Encapsulates the transactional booking agreement between a Patient and a Doctor. Contains metadata (appointmentId, appointmentDate, timeSlot, status [Pending, Confirmed, Rejected, Completed], notes) and operations (schedule(), confirmAppointment(), cancel(), reschedule()).
- **Schedule / Slot (Entity):** Represents the time-indexed availability matrix for a physician. Contains slotId, date, startTime, endTime, and isBooked status, with operations to markAvailable(), markBooked(), and lockSlot().
- **Notification (Support / Communication Service):** Automates the generation and transmission of multi-channel transactional notifications (SMS/Email). Contains notificationId, recipientId, messageBody, timeStamp, and deliveryStatus, with dispatchNotification() and notifyConfirmation().

## 1.2 Detailed Class Specifications & Structural Signatures

**• Class: User [Abstract]**

Attributes:
- userId: string
- fullName: string
- email: string
- passwordHash: string
- phone: string

Operations:
+ register(): bool
+ login(credentials): bool
+ logout(): void
+ updateProfile(): void

**• Class: Patient [Inherits from User]**

Attributes:
- bloodGroup: string
- medicalHistory: string
- emergencyContact: string

Operations:
+ searchDoctor(spec, date): List<Doctor>
+ requestAppointment(docId, slot): Appointment
+ cancelAppointment(appId): bool
+ viewAppointmentStatus(): List<Appointment>

**• Class: Doctor [Inherits from User]**

Attributes:
- doctorId: string
- specialization: string
- qualification: string
- consultationFee: double

Operations:
+ setAvailability(schedule): void
+ acceptAppointment(appId): bool
+ rejectAppointment(appId, reason): bool
+ viewAppointmentRequests(): List<Appointment>

- **Class: Appointment**

> Attributes:
>
> - appointmentId: string
> - patientId: string
> - doctorId: string
> - appointmentDate: Date
> - timeSlot: string
> - status: string [Pending, Confirmed, Rejected]
>
> Operations:
>
> + createAppointment(): bool
> + updateStatus(newStatus): void
> + sendConfirmation(): void

- **Class: Notification**

> Attributes:
>
> - notificationId: string
> - recipientEmail: string
> - message: string
> - sentTimestamp: DateTime
>
> Operations:
>
> + sendEmailNotification(): bool
> + sendSMSNotification(): bool

## 1.3 UML Class Diagram

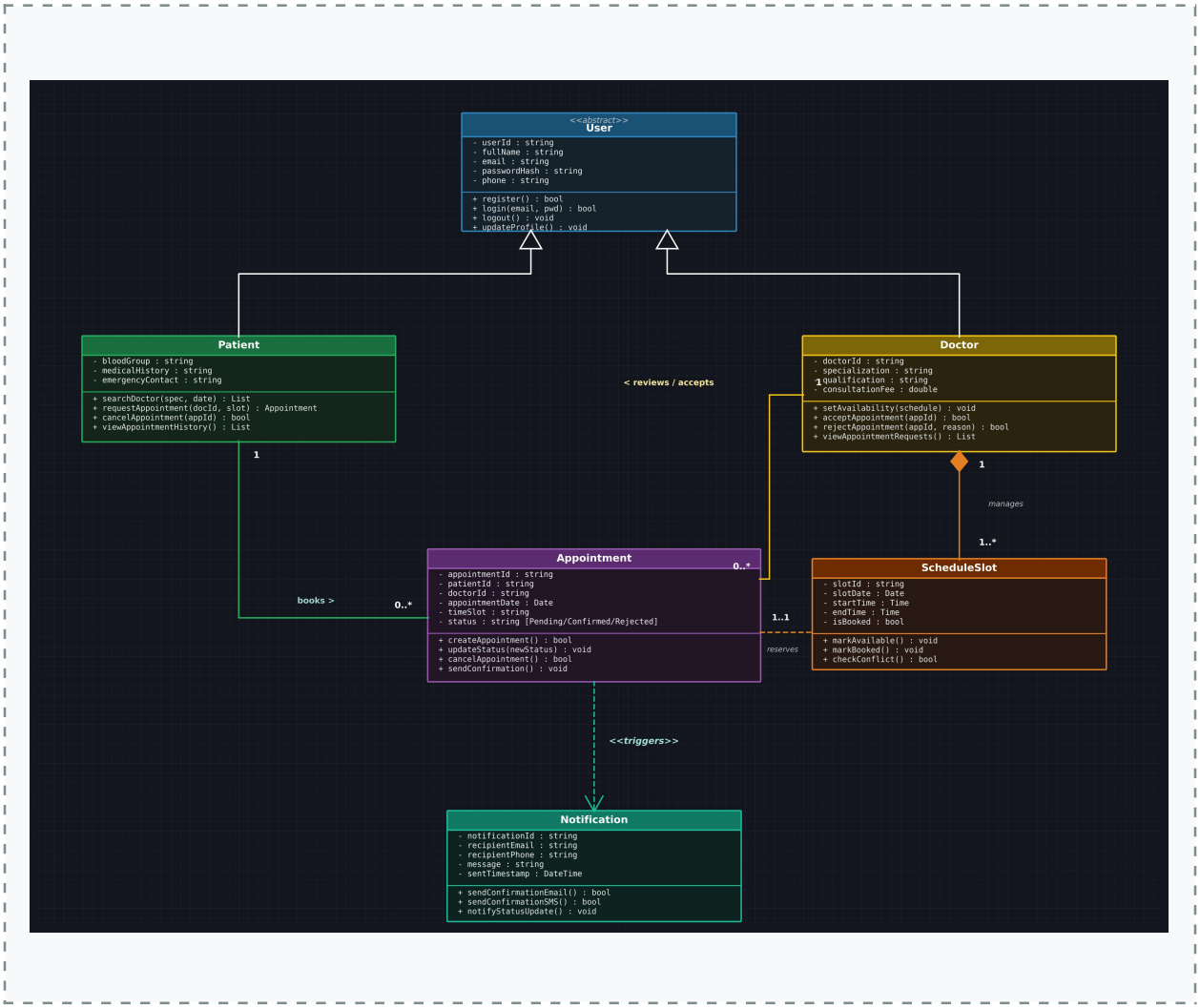



***Figure 1.1: Complete Class Diagram – Online Doctor Appointment Management System***

# Task 2: Inheritance Modeling & Implementation – Animal Hierarchy

**Problem Statement:** Implement a base superclass Animal with member functions eat() and sleep(). Create two derived subclasses Cat and Dog that inherit publicly from Animal. Cat implements its specialized behavior meow(), and Dog implements bark(). Model the class relationships using UML and provide clean, compiled C++ source code.

## 2.1 Class Structure & Inheritance Mapping

- **Animal (Base Class):** Encapsulates common state (name, age) and generic biological behaviors shared across all animals: eat() and sleep(). Virtual destructor included for standard polymorphic safety.
- **Dog (Derived Class):** Inherits publicly from Animal. In addition to inherited member methods, it provides the specialized vocalization bark().
- **Cat (Derived Class):** Inherits publicly from Animal. In addition to inherited member methods, it provides the specialized vocalization meow().

## 2.2 UML Class Diagram

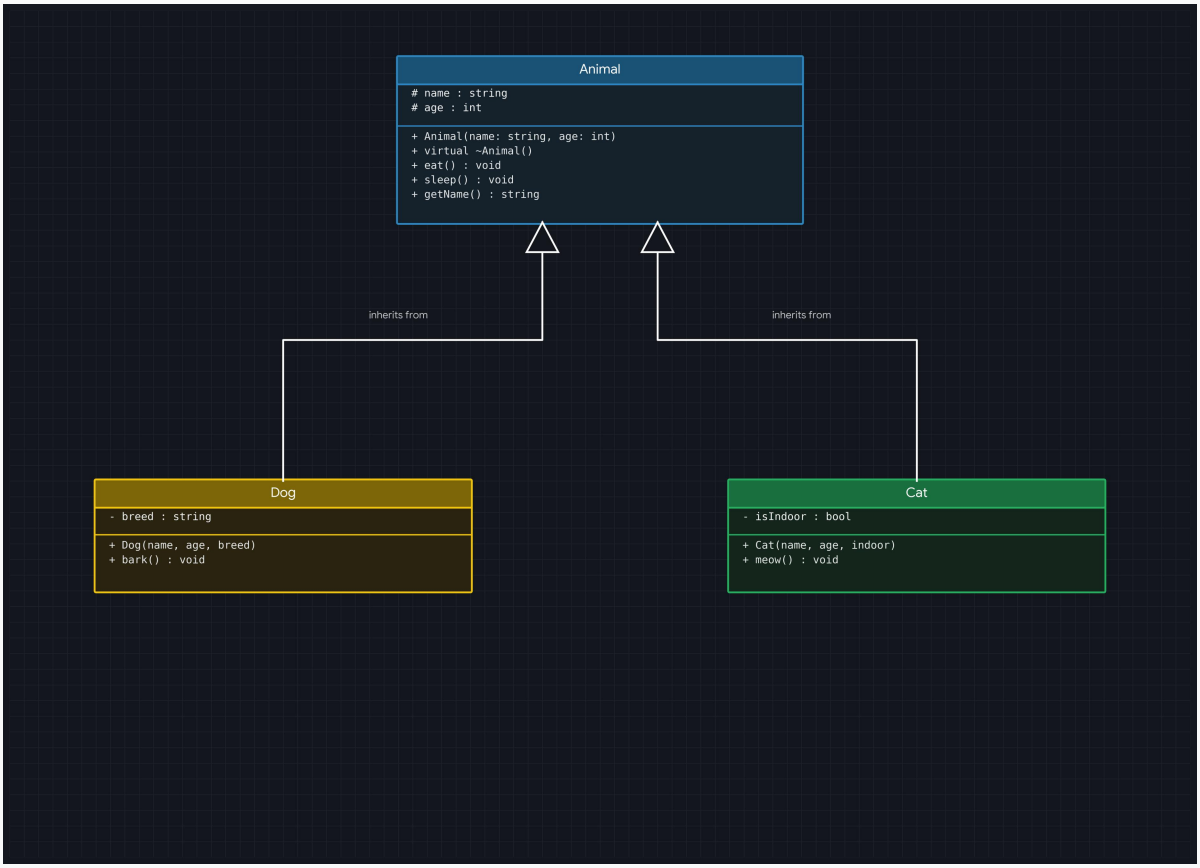


*Figure 2.1: UML Class Hierarchy Diagram – Animal Superclass with Cat and Dog Subclasses*

## 2.3 Object-Oriented C++ Implementation

```
#include <iostream>
#include <string>

// Base Superclass
class Animal {
protected:
    std::string name;
    int age;

public:
    // Parameterized Constructor
    Animal(const std::string& animalName = "Unknown Animal", int animalAge = 0)
        : name(animalName), age(animalAge) {}

    // Virtual Destructor for safe inheritance
    virtual ~Animal() = default;

    // Common Behaviors
    void eat() const {
        std::cout << name << " is eating healthy food." << std::endl;
    }

    void sleep() const {
        std::cout << name << " is sleeping peacefully." << std::endl;
    }

    std::string getName() const { return name; }
};

// Derived Subclass: Dog
class Dog : public Animal {
private:
    std::string breed;

public:
```

```
    Dog(const std::string& dogName, int dogAge, const std::string& dogBreed = "Mixed Breed")
        : Animal(dogName, dogAge), breed(dogBreed) {}

    // Specialized Behavior
    void bark() const {
        std::cout << name << " (" << breed << ") says: Woof! Woof!" << std::endl;
    }
};

// Derived Subclass: Cat
class Cat : public Animal {
private:
    bool isIndoor;

public:
    Cat(const std::string& catName, int catAge, bool indoor = true)
        : Animal(catName, catAge), isIndoor(indoor) {}

    // Specialized Behavior
    void meow() const {
        std::cout << name << " says: Meow! Purr..." << std::endl;
    }
};

int main() {
    std::cout << "=========================================" << std::endl;
    std::cout << "      Animal Inheritance Demonstration  " << std::endl;
    std::cout << "=========================================" << std::endl;

    // Instantiating Derived Objects
    Dog myDog("Buddy", 3, "Golden Retriever");
    Cat myCat("Luna", 2, true);

    std::cout << "\n--- Testing Dog Object ---" << std::endl;
    myDog.eat();    // Inherited from Animal
    myDog.sleep();  // Inherited from Animal
    myDog.bark();   // Specialized in Dog

    std::cout << "\n--- Testing Cat Object ---" << std::endl;
    myCat.eat();    // Inherited from Animal
```

```
    myCat.sleep();  // Inherited from Animal
    myCat.meow();   // Specialized in Cat

    return 0;
}
```

# Task 3: Polymorphism & Area Calculation – Shape Hierarchy

**Problem Statement:** Implement an abstract superclass Shape featuring pure virtual methods for area computation. Create two derived subclasses Triangle and Rectangle that inherit from Shape. Prompt the user for dimensions dynamically at runtime, calculate the respective geometric areas, display the results cleanly, draw the UML class diagram, and implement the complete C++ source code.

## 3.1 Class Architecture & Polymorphic Contracts

- **Shape (Abstract Base Class):** Defines the abstract interface for 2D geometric shapes. Encapsulates pure virtual method virtual double calculateArea() const = 0 and pure virtual method virtual void display() const = 0, enforcing strict polymorphism.
- **Triangle (Derived Class):** Implements Shape with private attributes base and height. Overrides calculateArea() using formula Area = 0.5 * base * height.
- **Rectangle (Derived Class):** Implements Shape with private attributes length and width. Overrides calculateArea() using formula Area = length * width.

## 3.2 UML Class Diagram

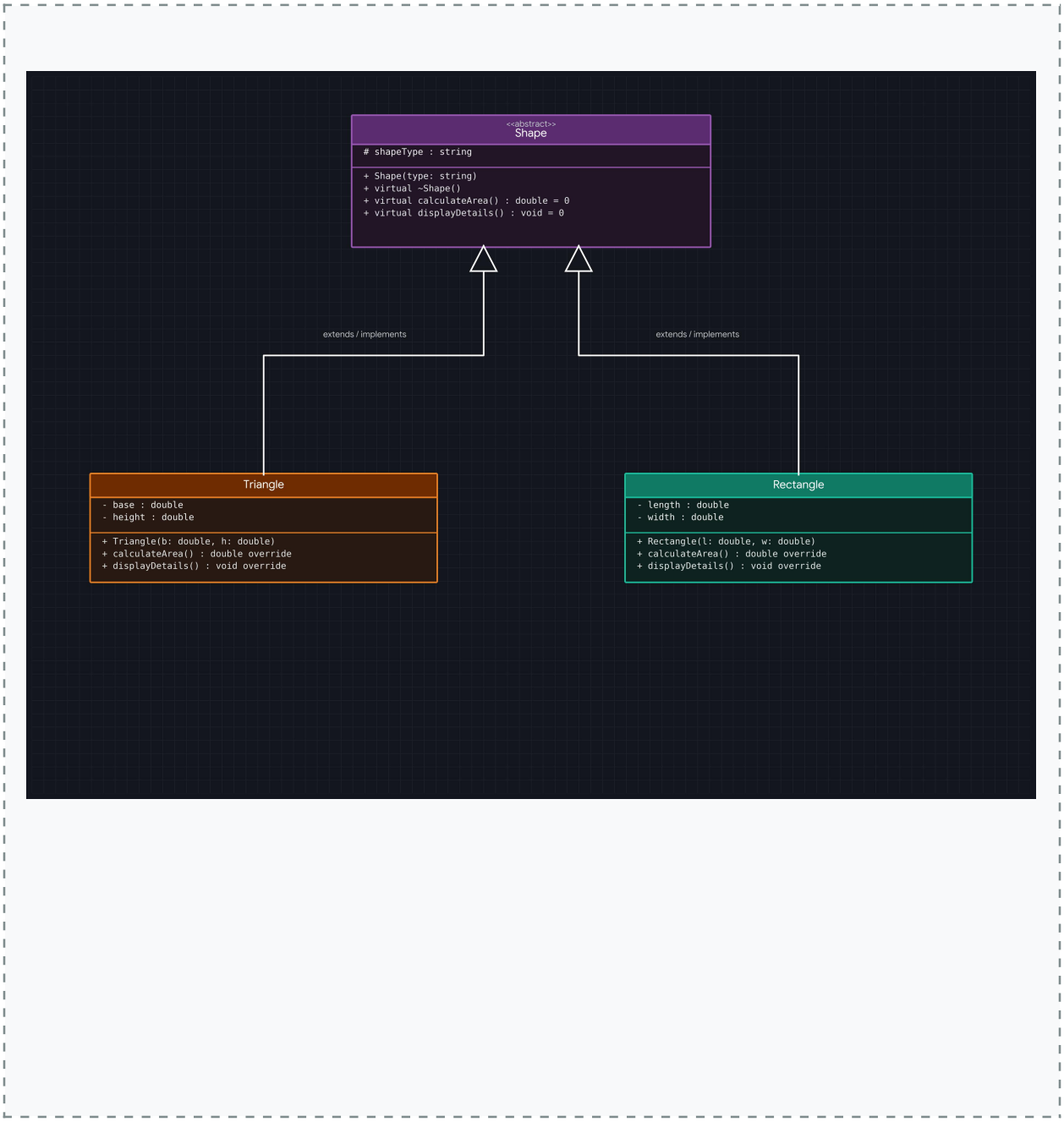



***Figure 3.1: UML Class Hierarchy Diagram – Shape Abstract Base Class with Triangle and Rectangle***

## 3.3 Object-Oriented C++ Implementation

```
#include <iostream>
#include <iomanip>
#include <string>

// Abstract Base Class: Shape
```

```
class Shape {
protected:
    std::string shapeType;

public:
    Shape(const std::string& type = "Generic Shape") : shapeType(type) {}
    virtual ~Shape() = default;

    // Pure Virtual Functions defining contract
    virtual double calculateArea() const = 0;
    virtual void displayDetails() const = 0;
};

// Derived Subclass: Triangle
class Triangle : public Shape {
private:
    double base;
    double height;

public:
    Triangle(double b = 0.0, double h = 0.0)
        : Shape("Triangle"), base(b), height(h) {}

    double calculateArea() const override {
        return 0.5 * base * height;
    }

    void displayDetails() const override {
        std::cout << "[Shape: " << shapeType << "]" << std::endl;
        std::cout << "  - Base:   " << base << " units" << std::endl;
        std::cout << "  - Height: " << height << " units" << std::endl;
        std::cout << "  - Area:   " << std::fixed << std::setprecision(2)
                  << calculateArea() << " sq units" << std::endl;
    }
};

// Derived Subclass: Rectangle
class Rectangle : public Shape {
private:
    double length;
```

```
    double width;

public:
    Rectangle(double l = 0.0, double w = 0.0)
        : Shape("Rectangle"), length(l), width(w) {}

    double calculateArea() const override {
        return length * width;
    }

    void displayDetails() const override {
        std::cout << "[Shape: " << shapeType << "]" << std::endl;
        std::cout << "  - Length: " << length << " units" << std::endl;
        std::cout << "  - Width:  " << width << " units" << std::endl;
        std::cout << "  - Area:   " << std::fixed << std::setprecision(2)
                  << calculateArea() << " sq units" << std::endl;
    }
};

int main() {
    std::cout << "================================================" << std::endl;
    std::cout << "       Shape Polymorphism & Area Calculator       " << std::endl;
    std::cout << "================================================" << std::endl;

    double b, h, l, w;

    // User input for Triangle
    std::cout << "\nEnter Triangle Dimensions:" << std::endl;
    std::cout << "  Enter base:   ";
    std::cin >> b;
    std::cout << "  Enter height: ";
    std::cin >> h;

    Triangle tri(b, h);

    // User input for Rectangle
    std::cout << "\nEnter Rectangle Dimensions:" << std::endl;
    std::cout << "  Enter length: ";
    std::cin >> l;
    std::cout << "  Enter width:  ";
```

```
    std::cin >> w;

    Rectangle rect(l, w);

    // Displaying Computed Results Polymorphically
    std::cout << "\n================ Computed Results ================" << std::endl;
    Shape* shapes[2];
    shapes[0] = &tri;
    shapes[1] = &rect;

    for (int i = 0; i < 2; ++i) {
        shapes[i]->displayDetails();
        std::cout << "--------------------------------------------------" << std::endl;
    }

    return 0;
}
```